

Classificador de Saúde Financeira de Pequeno Negócio

Plano Completo do Projeto — Versão 2

Estrutura corrigida · Com importação de CSV e predição de risco

Disciplina:	Introdução à Programação (Python) — UFCAT
Trabalho:	Aplicação Interativa para a Mostra UFCAT
Grupo:	4 integrantes
Duração:	4 semanas (1 mês)
Diferencial:	Predição de risco do negócio ao longo do tempo
Ferramentas:	Python · scikit-learn · pandas · tkinter · matplotlib

O que mudou nesta versão (leia primeiro)

Esta é a versão 2 do plano, corrigida depois de o grupo apontar pontos importantes. Três mudanças principais em relação à versão anterior:

- **Separação entre 'preparação' e 'aplicativo':** antes, o gerador de dados foi apresentado como 'Peça 1 do app', o que gerou confusão. Na verdade, ele faz parte de uma FASE DE PREPARAÇÃO (bastidor) que roda uma vez para criar o modelo. O aplicativo de verdade é outra coisa, e começa lendo os dados do usuário.
- **Duas portas de entrada de dados:** o usuário pode IMPORTAR um CSV com o histórico do negócio OU DIGITAR os números na tela. As duas opções convivem.
- **Predição de risco como diferencial central:** o app projeta a tendência do negócio ('se continuar assim, em X meses entra em risco'), com uma regra clara: só prevê quando há 2 meses ou mais de dados.

Por que essas mudanças importam: outros grupos estão fazendo projetos parecidos. A predição de risco é o que diferencia o de vocês. E a importação de CSV evita que o usuário tenha que digitar tudo à mão. São decisões do grupo, e este plano agora as reflete.

A grande ideia: o projeto tem DUAS fases

Este é o conceito mais importante de todo o documento. Entender isso resolve a confusão sobre 'por onde começar'. O projeto se divide em duas fases bem diferentes, que mexem com dados diferentes.

Fase 1 — Preparação (o bastidor)

Esta fase acontece UMA vez, durante o desenvolvimento, e serve para criar o 'cérebro' do modelo. Ela mexe com DADOS DE TREINO: negócios fictícios que ensinam a IA. Pense nela como uma oficina onde vocês fabricam uma peça (o modelo treinado) que depois será encaixada no aplicativo. O usuário final NUNCA vê esta fase — ela é só dos desenvolvedores.

- **Gerar dados de treino** — criar os negócios fictícios variados (gerar_dados.py)
- **Treinar o modelo** — fazer a IA aprender com esses dados e salvar o cérebro treinado (treinar_modelo.py)

Fase 2 — O Aplicativo (o que o usuário usa)

Esta é a fase que roda toda vez que alguém usa o programa. Ela mexe com DADOS DO USUÁRIO: os números reais do negócio da pessoa. O fluxo começa com a pessoa fornecendo os dados (importando um CSV ou digitando), e termina com o diagnóstico e a predição na tela.

- **Entrada de dados** — o usuário importa um CSV OU digita os números (interface.py)
- **Cálculo e análise** — calcular indicadores e achar o fator crítico por setor (analise.py)
- **Classificação** — o modelo treinado diz a saúde atual (usa o cérebro da Fase 1)
- **Predição** — projetar o risco futuro, se houver 2+ meses (predicao.py)
- **Juntar tudo** — amarrar o fluxo completo (main.py)

A confusão resolvida: o gerar_dados.py NÃO é o 'passo 1 do app'. Ele é o passo 1 da PREPARAÇÃO. O app de verdade começa quando o usuário fornece seus dados. São dois mundos: dados de treino (fictícios, para ensinar) e dados do usuário (reais, para avaliar). Não confunda os dois.

Como o aplicativo funciona (o fluxo completo)

Quando alguém usa o app, este é o caminho que os dados percorrem, do início ao fim:

Passo	O que acontece
1. Entrada	A pessoa escolhe: importar um CSV com o histórico OU digitar os números de um mês na tela
2. Leitura	O programa lê esses dados (de qualquer uma das duas portas) e os organiza
3. Cálculo	Calcula os indicadores: margem, peso do custo, liquidez, endividamento
4. Classificação	O modelo treinado diz se cada mês está saudável, em atenção ou em risco
5. Fator crítico	Compara os indicadores com a faixa do SETOR e aponta o maior problema
6. Predição	SE houver 2+ meses, projeta a tendência ('se continuar assim, em X meses...')
7. Exibição	Mostra tudo na tela: saúde atual, fator crítico, predição e um gráfico
8. Salvar	Guarda os dados no histórico do negócio (para uso futuro e mais predições)

A regra da predição (importante para a honestidade)

A predição só aparece quando o negócio tem DOIS MESES OU MAIS de dados. O motivo é simples e defensável: tendência é direção, e direção precisa de pelo menos dois pontos para existir. Com um mês só, há apenas um ponto isolado, e não dá para saber se as coisas estão melhorando ou piorando.

Situação	O que o app faz
1 mês de dados	Mostra só a análise e a classificação daquele mês. Avisa que precisa de mais um mês para prev
2 meses ou mais	Mostra a análise E a predição: 'se continuar nessa direção, em X meses o negócio entra em risco

Resposta pronta para a banca: 'Por que a predição precisa de 2 meses? Porque tendência é direção, e direção só existe com no mínimo dois pontos no tempo. Com um mês, seria inventar uma tendência que os dados não mostram — e nós evitamos afirmar mais do que os dados permitem.'

Todas as peças do projeto

Juntando as duas fases, o projeto tem 7 peças de código. As 2 primeiras são da preparação; as 5 seguintes formam o aplicativo. A ordem de construção segue a dependência entre elas.

Fase 1 — Preparação

#	Arquivo	O que faz
P1	gerar_dados.py	Cria os negócios fictícios de treino e salva no treino.csv
P2	treinar_modelo.py	Lê o treino.csv, treina o modelo e salva o cérebro (modelo.pkl)

Fase 2 — Aplicativo

#	Arquivo	O que faz
A1	analise.py	Calcula indicadores e acha o fator crítico por setor
A2	predicao.py	Projeta a tendência de risco (regra dos 2 meses)
A3	interface.py	As telas: importar CSV, digitar, e ver resultados
A4	dados_usuario.py	Lê o CSV importado e salva o histórico do usuário
A5	main.py	Junta tudo num programa que roda; tem a classe Negocio

Sobre o tamanho: este projeto é mais ambicioso que a versão enxuta. Por isso é importante saber o que é NÚCLEO (precisa funcionar) e o que é DIFERENCIAL (predição, importar CSV). Se o tempo apertar, protejam o núcleo primeiro. O diferencial pode virar 'trabalho futuro' sem comprometer a entrega.

Núcleo essencial vs. Diferencial (rede de segurança)

Para o grupo não se afogar, aqui está o que é indispensável e o que é o 'algo a mais'. Se faltar tempo, garantam o núcleo primeiro.

Núcleo essencial — TEM que funcionar

- Gerar dados de treino e treinar o modelo (Fase 1 inteira)
- Digitar os dados de UM negócio na tela
- Calcular os indicadores e classificar a saúde (saudável/atenção/risco)
- Mostrar o fator crítico (qual indicador está pior)
- Salvar e mostrar o histórico em CSV
- A classe Negocio (exigida pelo PDF)

Diferencial — o que destaca vocês

- **Importar CSV** — carregar vários meses de uma vez, sem digitar
- **Predição de risco** — projetar a tendência com a regra dos 2 meses
- **Setores** — faixas de referência diferentes por ramo de negócio
- **Gráfico de evolução** — visualizar a saúde ao longo do tempo

Estratégia: construam o núcleo nas primeiras semanas. Quando ele estiver sólido e compreendido, adicionem o diferencial camada por camada. Assim vocês têm sempre algo funcionando para entregar, e o diferencial vai sendo somado por cima de uma base firme.

Divisão por pessoa

Cada integrante lidera uma frente, mas todos estudam o código de todos — porque a nota individual vem da arguição, onde cada um precisa saber explicar o projeto inteiro.

Pessoa	Frente principal	Responsabilidades
Pessoa A	Modelo (Fase 1)	Gerar dados de treino e treinar o classificador (gerar_dados + treinar_modelo)
Pessoa B	Dados e Predição	Ler CSV do usuário, salvar histórico, e a predição de tendência (dados_usuario + pr
Pessoa C	Interface	As telas: importar, digitar, e exibir resultados (interface)
Pessoa D	Análise e Integração	Indicadores, setores, juntar tudo no main, classe Negocio, e documentação (analise

Trabalho em paralelo: ninguém precisa esperar o outro. A Pessoa C monta as telas com dados de mentira enquanto o modelo não está pronto. A Pessoa A trabalha na preparação enquanto a interface é construída. Combinem o 'contrato' (que dados passam de uma peça para outra) logo no início, e cada um toca sua parte. A integração final junta tudo.

Tarefas detalhadas por pessoa

Pessoa A — Modelo (Fase 1)

- Estudar o gerar_dados.py (já escrito) até entender cada bloco
- Rodar o gerar_dados.py e conferir o treino.csv gerado
- Escrever o treinar_modelo.py: ler o CSV com pandas
- Transformar a coluna 'setor' (texto) em número para o modelo entender
- Criar e treinar um classificador (árvore de decisão) com .fit()
- Medir a acurácia e salvar o modelo treinado (modelo.pkl)
- Saber explicar: o que é treino vs. uso, e por que isso é Machine Learning

Pessoa B — Dados do Usuário e Predição

- Escrever o dados_usuario.py: ler um CSV importado pelo usuário
- Validar se o CSV tem as colunas certas (tratar erro se não tiver)
- Salvar e carregar o histórico do negócio em CSV
- Escrever o predicacao.py: calcular a tendência entre os meses
- Implementar a regra dos 2 meses (sem histórico suficiente, não prevê)
- Montar a mensagem de predição em linguagem honesta ('se continuar assim...')
- Saber explicar: por que a predição precisa de 2 meses

Pessoa C — Interface

- Aprender o básico de tkinter (janela, campos, botões, rótulos)
- Criar a tela inicial com as DUAS opções: importar CSV ou digitar
- Montar o botão de importar (abre o seletor de arquivos do sistema)
- Montar a tela de digitar, com todos os campos do negócio
- Montar a tela de resultado, com cor conforme a classificação
- Exibir o fator crítico e a predição de forma clara
- Começar com dados de mentira, sem esperar as outras peças

Pessoa D — Análise e Integração

- Escrever o analise.py: funções que calculam cada indicador
- Criar a tabela de faixas saudáveis por setor (pesquisar as referências)
- Escrever a lógica do fator crítico (comparar com a faixa do setor)
- Escrever o main.py que junta todas as peças
- Criar a classe Negocio (vai precisar aprender POO do zero)
- Adicionar o tratamento de erros (try/except)
- Tocar a documentação técnica e o roteiro de apresentação

Instruções para o Claude Code (organização do repositório)

Esta seção é dirigida ao Claude Code, que vai organizar o repositório no GitHub. O grupo vai importar este PDF e fornecer o link do repositório para que o Claude Code monte a estrutura abaixo.

O que o Claude Code deve criar no repositório

Claude Code: por favor, organize o repositório fornecido com a estrutura de pastas e as branches abaixo. NÃO escreva o código das peças ainda — apenas crie a estrutura, os arquivos vazios com um comentário de cabeçalho explicando o propósito de cada um, e o esqueleto organizacional. O grupo vai preencher cada arquivo estudando junto, peça por peça.

Estrutura de pastas a criar

```
projeto-saude-financeira/  
|-- main.py                (A5 - junta tudo + classe Negocio)  
|-- requirements.txt       (pandas, scikit-learn, matplotlib)  
|-- README.md              (descrição do projeto)  
|-- preparacao/            (FASE 1 - bastidor)  
    |-- gerar_dados.py     (P1)  
    |-- treinar_modelo.py  (P2)  
|-- app/                   (FASE 2 - aplicativo)  
    |-- analise.py         (A1)  
    |-- predicao.py        (A2)  
    |-- interface.py       (A3)  
    |-- dados_usuario.py   (A4)  
|-- data/  
    |-- treino.csv         (gerado pela P1)  
    |-- exemplo_usuario.csv (modelo para o usuário importar)  
|-- docs/                  (documentação técnica)
```

Branches (abas) a criar — uma por frente de trabalho

Para o grupo trabalhar em paralelo sem conflitos, criar uma branch para cada pessoa, além da main:

Branch	Para quem	O que será desenvolvido nela
main	Todos	Versão estável; recebe as partes prontas via merge
fase1-modelo	Pessoa A	gerar_dados.py e treinar_modelo.py
dados-predicao	Pessoa B	dados_usuario.py e predicao.py
interface	Pessoa C	interface.py
analise-main	Pessoa D	analise.py, main.py e a classe Negocio

Claude Code: em cada arquivo vazio, inclua um comentário de cabeçalho (docstring) explicando: o propósito do arquivo, qual peça ele é, quem é o responsável, e o que ele recebe e devolve. Crie também o README.md com a descrição do projeto e as instruções de instalação. Configure o .gitignore para ignorar arquivos temporários do Python (__pycache__, etc.). NÃO implemente a lógica das peças — o grupo fará isso estudando cada uma.

Cronograma das 4 semanas

Com cerca de 2 a 3 horas por semana por integrante. As três entregas exigidas pelo PDF estão marcadas. O núcleo vem primeiro; o diferencial é somado depois.

Semana	Foco	Entrega
Semana 1	Preparar ambiente, criar repositório (Claude Code), Fase 1 (GUI + Fases 1 + Proposta (PDF)	ENTREGA 1
Semana 2	Núcleo do app: análise, classificação, digitar dados, classe (desenvolvimento)	Núcleo
Semana 3	Diferencial: importar CSV, predição de risco, interface completa, integração	ENTREGA 2
Semana 4	Polir, testar, gráficos, documentação, pôster, ENSAIAR apresentação	ENTREGA 3

Lembrete sobre a ambição: este projeto é maior que o básico, porque vocês querem se destacar e têm boas razões. A rede de segurança é a divisão núcleo/diferencial: se o tempo apertar, entreguem o núcleo sólido e cite o resto como evolução. Melhor um projeto menor que funciona e que vocês entendem do que um grande que trava na arguição.

Como o projeto cumpre os requisitos do PDF

Requisito do PDF	Onde aparece
Mínimo 3 módulos separados	Temos 7 arquivos separados
Variáveis e tipos	Indicadores, números do negócio
if/elif/else	Comparação com faixas; regra dos 2 meses
Laços (for/while)	Geração de dados; iteração nos meses
Funções e modularização	Base de todas as peças
Listas, dicionários, tuplas	Negócios, faixas por setor, históricos
Leitura/escrita de arquivos	CSV de treino, CSV do usuário, importar CSV
try/except	Validação do CSV importado e de entradas
POO (1 classe)	Classe Negocio (main.py)
Machine Learning básico	Classificador scikit-learn (Fase 1)
Interface de usuário	Telas em tkinter (importar e digitar)

Bônus que o projeto já mira (até 5 pts extras)

- **Importar CSV** e manipular arquivos de forma avançada
- **Predição de tendência** — vai além do básico pedido
- Possível integração com API externa (ex: taxa de juros) como evolução futura

Sobre o uso de IA — honestidade que vale pontos

O PDF permite IA como apoio, desde que o grupo compreenda e saiba explicar todo o código, e desde que o uso seja declarado. O plano de vocês — estudar cada linha até entender — é exatamente o uso correto. Declarem na documentação que usaram IA como ferramenta de aprendizado e apoio, e que cada integrante compreende o código. Isso é honesto, cumpre a regra, e demonstra maturidade.

O que decide a nota: a arguição individual. Não importa quem escreveu cada linha — importa quem ENTENDE. Estudem juntos, expliquem uns para os outros, cheguem na Mostra sabendo defender cada parte. É assim que vocês fazem o melhor trabalho da sala.